Frontend con React

Aplicaciones Interactivas

UADE

Lista de tareas pendientes

Vamos a implementar una aplicación simple con **React** + **Spring**.

La aplicación nos permitirá:

- Listar tareas
- Agregar tareas nuevas
- Modificar tareas existentes
- Completar tarea pendiente

Backend con Spring

Backend con Spring

Ya tenemos un backend mínimo funcionando con **Spring**.

Controller

TareaController.java

src > main > java > com > uade > todolist > controller > TareaController.java > Language Support for Java(TM) by Red Hat > TareaContr

```java
import java.util.List;

@RestController
@RequestMapping("/api/tareas")
public class TareaController {

    private final TareaService tareaService;

    public TareaController(TareaService tareaService) {
        this.tareaService = tareaService;
    }

    @GetMapping
    public List<Tarea> listarTareas() {···

    @GetMapping("/{id}")
    public ResponseEntity<Tarea> buscarTarea(@PathVariable Long id) {···

    @PostMapping
    public Tarea crearTarea(@RequestBody Tarea tarea) {···

    @PutMapping("/{id}")
    public ResponseEntity<Tarea> modificarTarea(@PathVariable Long id, @RequestBody Tarea tarea) {···

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminarTarea(@PathVariable Long id) {···
}
```

Model

Tarea.java

src > main > java > com > uade > todolist > model > Tarea.java > ...

```java

@Data
@Entity
public class Tarea {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String titulo;

    private String descripcion;

    @Enumerated(EnumType.STRING)
    private TipoTarea tipo;

    private boolean completada;
}

```

TipoTarea.java

src > main > java > com > uade > todolist > model > TipoTarea.java > ...

```java
package com.uade.todolist.model;

public enum TipoTarea {
    TRABAJO,
    ESTUDIO,
    PERSONAL
}

```

Configuración Inicial React

Configuración Inicial



Node.js

Es un entorno de ejecución que permite ejecutar JavaScript fuera del navegador, por ejemplo, desde la consola o en un servidor.

React no necesita Node.js para funcionar en el navegador, pero lo usamos durante el desarrollo para ejecutar herramientas como **Vite**, instalar dependencias y preparar el código de la aplicación.

Descarga del sitio oficial

Configuración Inicial

Vite

Herramienta de compilación y desarrollo frontend diseñada para acelerar la creación de aplicaciones web.

```
npm create vite@latest nombre-del-proyecto -- --template react
```

Una vez creado, en la carpeta del proyecto:

```
npm install
```

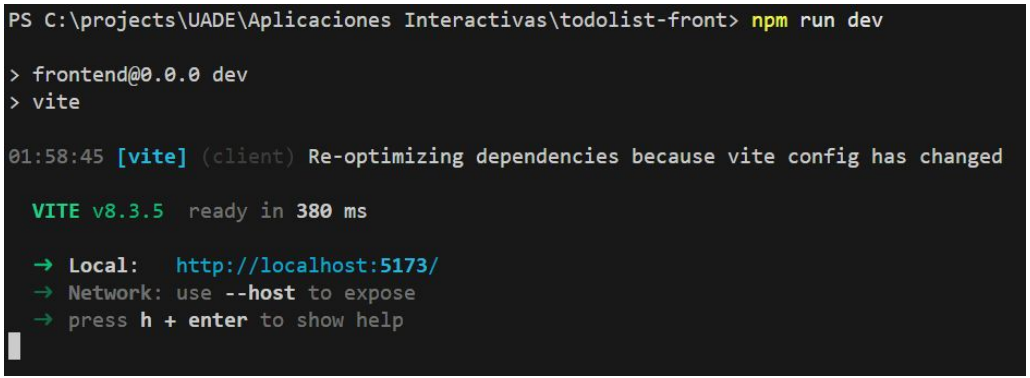
Configuración Inicial

Servidor de desarrollo con Vite

iniciar el servidor de desarrollo de Vite, que nos permite ejecutar y visualizar nuestra aplicación React en el navegador mientras programamos, con actualizaciones automáticas al guardar cambios.

npm run dev

http://localhost:5173



localhost:5173



Get started

Edit `src/App.jsx` and save to test `HMR`

Count is 0

Documentation

Your questions, answered

Explore Vite | Learn more

Connect with us

Join the Vite community

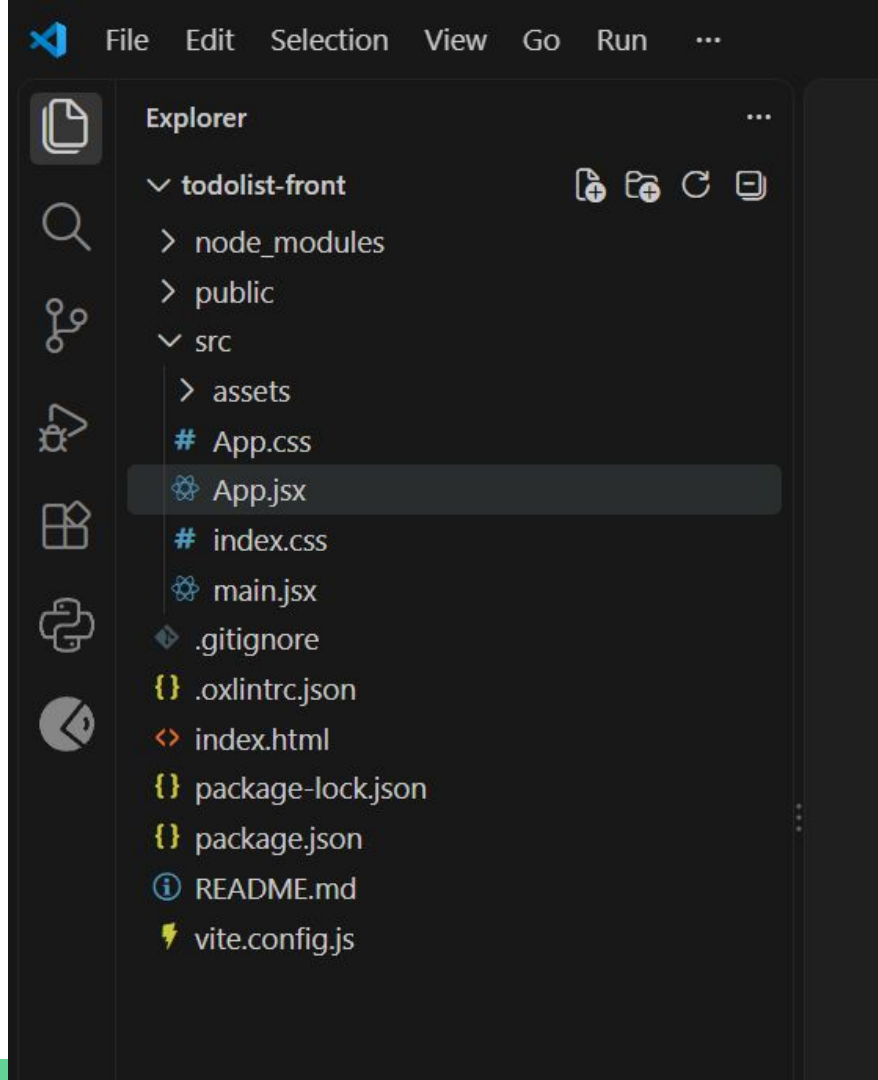
GitHub | Discord | X.com | Bluesky

Configuración Inicial

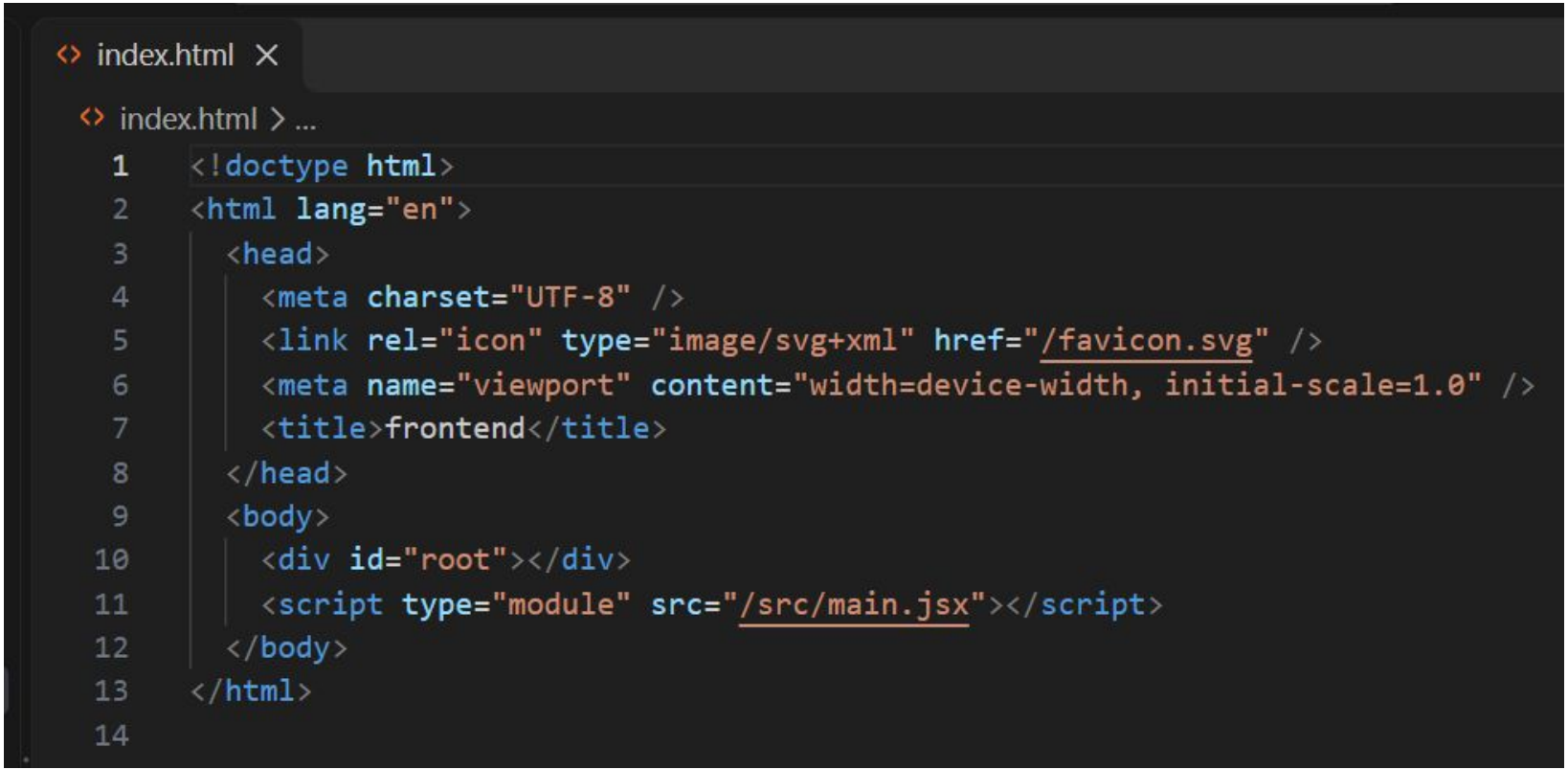
Analicemos la estructura básica de un proyecto **React**.

Estructura básica

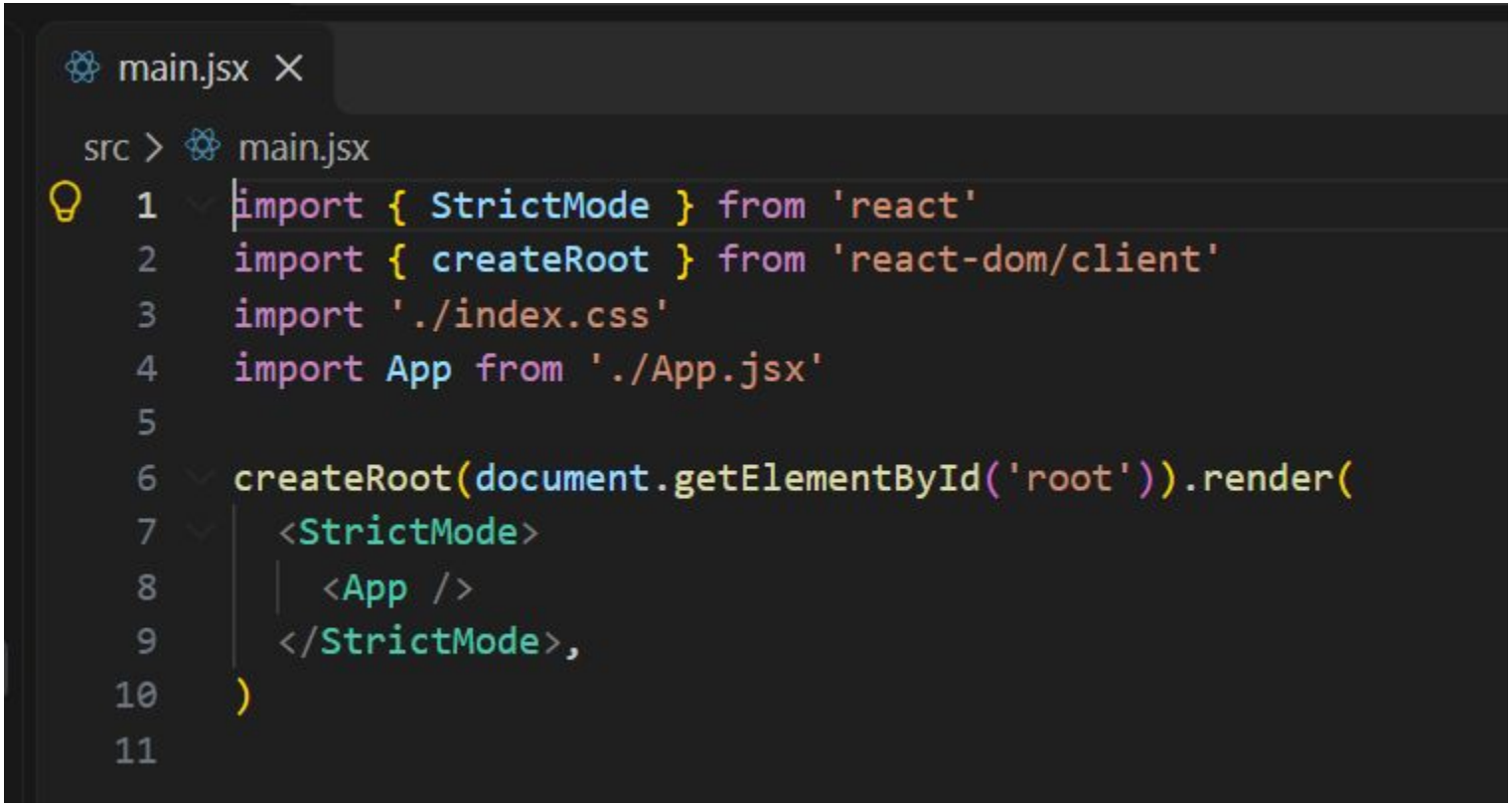
- node_modules
- src
  - App.css
  - App.jsx
  - index.css
  - main.jsx
- index.html
- package.json



index.html


index.html
index.html > ...
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
6 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7 <title>frontend</title>
8 </head>
9 <body>
10
11 <script type="module" src="/src/main.jsx"></script>
12 </body>
13 </html>
14

main.jsx


main.jsx
src > main.jsx
1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import './index.css'
4 import App from './App.jsx'
5
6 createRoot(document.getElementById('root')).render(
7 <StrictMode>
8 <App />
9 </StrictMode>,
10)
11

App.jsx

App.jsx

src > App.jsx > App

```jsx
import { useState } from 'react'
import heroImg from './assets/hero.png'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

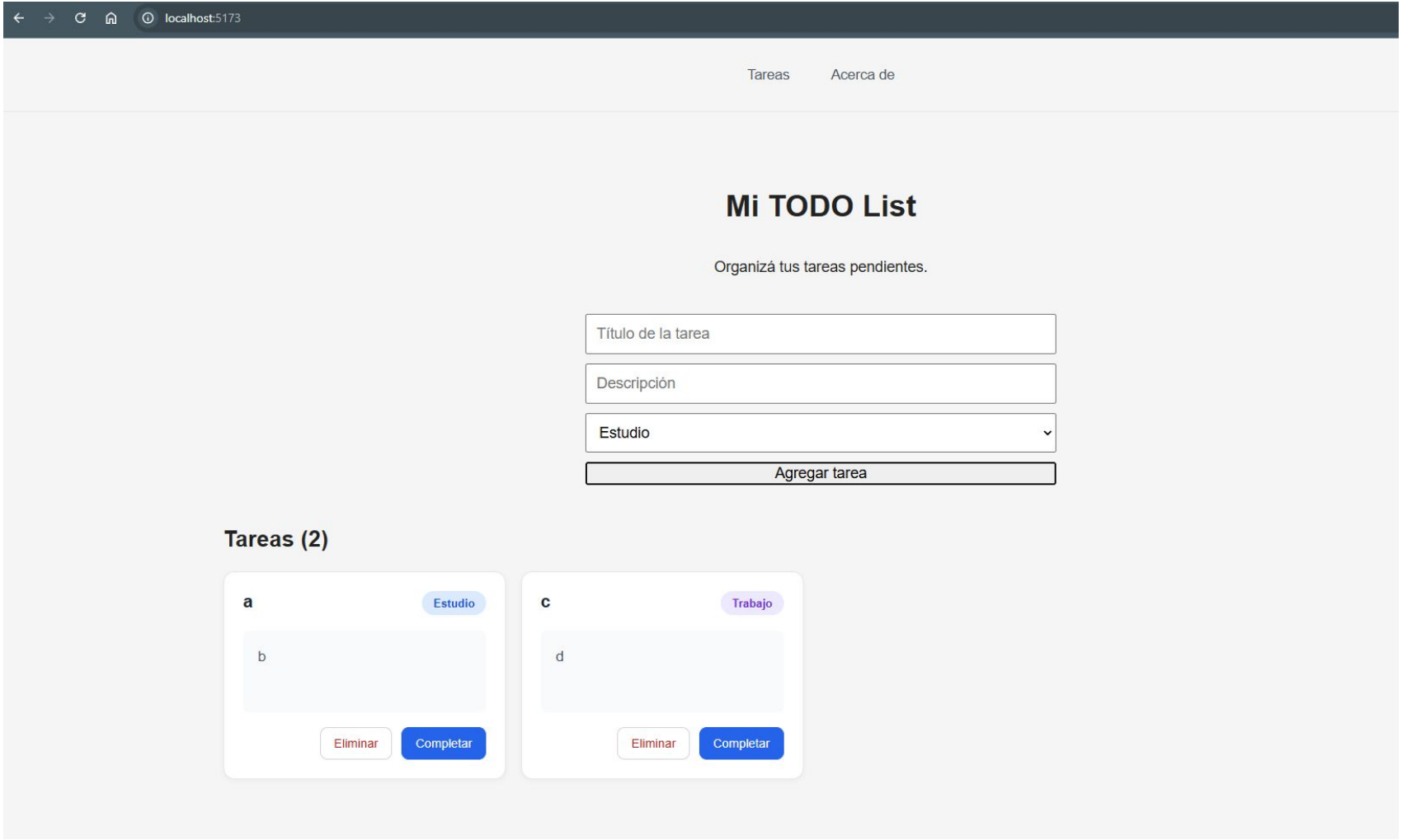
  return (
    <>
      <section id="center">
        <div className="hero">
          <img src={heroImg} className="base" width="170" height="179" alt="" />
          <img src={reactLogo} className="framework" alt="React logo" />
          <img src={viteLogo} className="vite" alt="Vite logo" />
        </div>
        <div>
          <h1>Get started</h1>
          <p>
            Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
          </p>
```

TODO list

Analicemos la app y sus componentes principales

Github

https://github.com/lpueyoUADE/TodoList


localhost:5173
Tareas
Acerca de
Mi TODO List
Organizá tus tareas pendientes.
Título de la tarea
Descripción
Estudio
Agregar tarea
Tareas (2)
a
Estudio
b
Eliminar
Completar
c
Trabajo
d
Eliminar
Completar

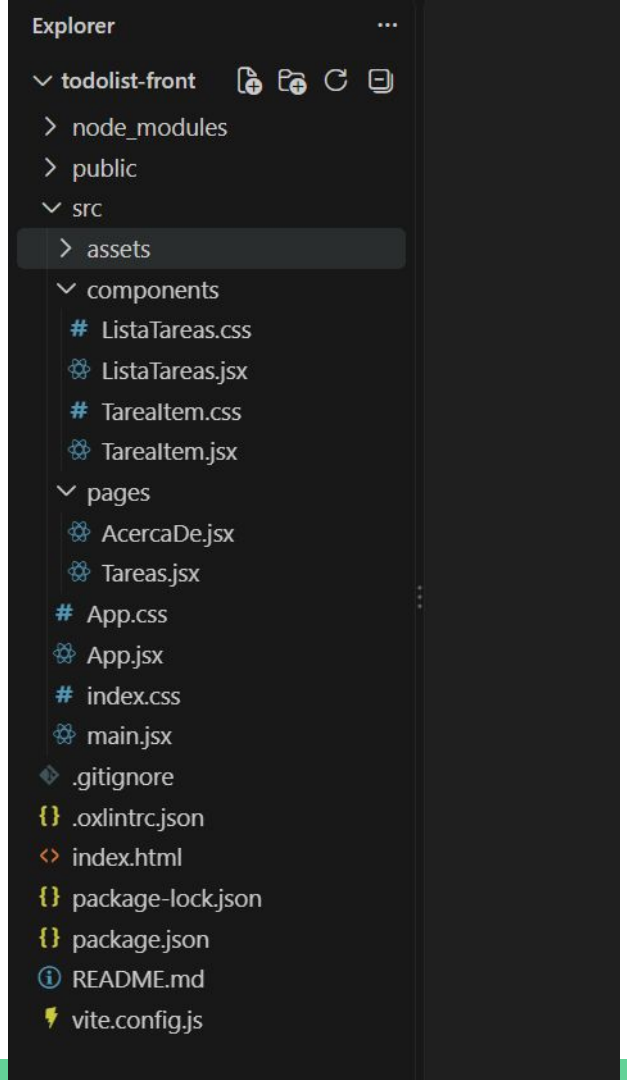
Estructura del proyecto

Estructura de proyecto

La estructura básica de react se mantiene.

Aparecen nuevas carpetas:

- **Components**
- **Pages**



App.jsx

App.jsx configura la navegación de la aplicación con **React Router**, definiendo las rutas y el menú para acceder a las páginas de Tareas y Acerca de.

App.jsx

src > App.jsx > ...

```jsx
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom'
import './App.css'
import Tareas from './pages/Tareas'
import AcercaDe from './pages/acercaDe'

function App() {
  return (
    <BrowserRouter>
      <nav className="navegacion">
        <Link to="/">Tareas</Link>
        <Link to="/acerca-de">Acerca de</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Tareas />} />
        <Route path="/acerca-de" element={<AcercaDe />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App
```

Páginas

Tareas.jsx

gestiona el estado de las tareas, permite crearlas, completarlas y eliminarlas, y se comunica con la **API REST** para mantener los datos actualizados.

Tareas.jsx

src > pages > Tareas.jsx > Tareas

```jsx
93
94  return (
95    <main className="contenedor">
96      <h1>Mi TODO List</h1>
97      <p>Organizá tus tareas pendientes.</p>
98
99      <form onSubmit={agregarTarea}>
100       <input
101         type="text"
102         placeholder="Título de la tarea"
103         value={titulo}
104         onChange={event => setTitulo(event.target.value)}
105       />
106
107       <input
108         type="text"
109         placeholder="Descripción"
110         value={descripcion}
111         onChange={event => setDescripcion(event.target.value)}
112       />
113
114       <select
115         value={tipo}
116         onChange={event => setTipo(event.target.value)}
117       >
118         <option value="TRABAJO">Trabajo</option>
119         <option value="ESTUDIO">Estudio</option>
120         <option value="PERSONAL">Personal</option>
121       </select>
122
123       <button type="submit">Agregar tarea</button>
```

AcercaDe.jsx

Presenta una descripción del trabajo práctico, las tecnologías utilizadas y los principales conceptos de React aplicados en el desarrollo de la aplicación.

AcercaDe.jsx

src > pages > AcercaDe.jsx > AcercaDe

```jsx

function AcercaDe() {
    return (
        <main className="contenedor">
            <h1>Acerca de este proyecto</h1>

            <p>
                Este proyecto consiste en una aplicación de gestión de tareas
                desarrollada como trabajo práctico. Permite crear, listar,
                completar y eliminar tareas, organizadas por categoría.
            </p>

            <h2>Tecnologías utilizadas</h2>

            <ul>
                <li>React para construir la interfaz de usuario.</li>
                <li>Spring Boot para desarrollar la API REST.</li>
                <li>H2 como base de datos para almacenar las tareas.</li>
            </ul>

            <h2>Conceptos de React aplicados</h2>

            <ul>
                <li>
                    <strong>Componentes:</strong> dividimos la interfaz en partes
                    reutilizables, como ListaTareas y TareaItem.
                </li>
                <li>
                    <strong>Props:</strong> enviamos datos y funciones entre
                    componentes.
                </li>
```

Componentes

ListaTareas.jsx

recibe la lista de tareas mediante **props** y utiliza el método map() para generar una tarjeta por cada tarea, mostrando también la cantidad total y un mensaje cuando no hay tareas.

ListaTareas.jsx

src > components > ListaTareas.jsx > ...

```jsx
import './ListaTareas.css'
import TareaItem from './TareaItem'

function ListaTareas({ tareas, onCambiarEstado, onEliminar }) {
  return (
    <section>
      <h2>Tareas ({tareas.length})</h2>

      {tareas.length === 0 && <p>No hay tareas.</p>}

      <div className="lista-tareas">
        {tareas.map(tarea => (
          <TareaItem
            key={tarea.id}
            tarea={tarea}
            onCambiarEstado={onCambiarEstado}
            onEliminar={onEliminar}
          />
        ))}
      </div>
    </section>
  )
}

export default ListaTareas
```

Props

Las props (properties) en React son datos que un componente recibe de su componente **padre** para mostrarlos o utilizarlos. Permiten que los componentes se comuniquen entre sí y sean reutilizables.

Props

Tareas.jsx

src > pages > Tareas.jsx > default

```jsx
          >
            <option value="TRABAJO">Trabajo</option>
            <option value="ESTUDIO">Estudio</option>
            <option value="PERSONAL">Personal</option>
          </select>

          <button type="submit">Agregar tarea</button>
        </form>

        <ListaTareas
          tareas={tareas}
          onCambiarEstado={cambiarEstado}
          onEliminar={eliminarTarea}
        />
      </main>
    )
  }

export default Tareas
```

ListaTareas.jsx

src > components > ListaTareas.jsx > ...

```jsx
import './ListaTareas.css'
import TareaItem from './TareaItem'

function ListaTareas({ tareas, onCambiarEstado, onEliminar }) {
  return (
    <section>
      <h2>Tareas ({tareas.length})</h2>

      {tareas.length === 0 && <p>No hay tareas.</p>}

      <div className="lista-tareas">
```

TarealItem.jsx

Representa cada tarea individual mediante una tarjeta que muestra su título, descripción y categoría, e incluye botones para completarla, reabrirla o eliminarla mediante funciones recibidas por props.

TarealItem.jsx

src > components > TarealItem.jsx > ...

```jsx

import './TareaItem.css'

function TareaItem({ tarea, onCambiarEstado, onEliminar }) {
  return (
    <article className={`tarea ${tarea.completada ? 'tarea-completada' : ''}`}>
      <header className="tarea-header">
        <h3>{tarea.titulo}</h3>

        <span className={`tarea-tipo tipo-${tarea.tipo.toLowerCase()}`}>
          {tarea.tipo.charAt(0) + tarea.tipo.slice(1).toLowerCase()}
        </span>
      </header>

      <div className="tarea-descripcion">
        {tarea.descripcion || 'Sin descripción'}
      </div>

      <footer className="tarea-footer">
        <button
          className="boton-eliminar"
          onClick={() => onEliminar(tarea.id)}
        >
          Eliminar
        </button>

        <button
          className="boton-completar"
          onClick={() => onCambiarEstado(tarea)}
        >
          {tarea.completada ? 'Reabrir tarea' : 'Completar'}
```

Hooks

funciones de React

Hooks

Los Hooks de React son funciones especiales que permiten agregar funcionalidades a los componentes, como **manejar datos que cambian** o **ejecutar acciones** cuando se carga una página.

https://www.w3schools.com/react/react_hooks.asp

Hooks

useState

Manejo del estado. Permite guardar datos que pueden cambiar y actualizar la interfaz cuando estos datos cambian.

https://www.w3schools.com/react/react_usestate.asp

UseState

- **titulo**: contiene el valor actual.
- **setTitulo**: función que permite actualizar ese valor.
- **useState**(''): inicializa el estado con una cadena vacía.

Cuando el usuario escribe en el campo de título, usamos **setTitulo** para guardar el nuevo valor. React actualiza la interfaz para reflejarlo.

Tareas.jsx

src > pages > Tareas.jsx > Tareas > useEffect() callback > then() callback

```jsx
import { useEffect, useState } from 'react'
import ListaTareas from '../components/ListaTareas'

const API_URL = 'http://localhost:8080/api/tareas'

function Tareas() {
  const [tareas, setTareas] = useState([])
  const [titulo, setTitulo] = useState('')
  const [descripcion, setDescripcion] = useState('')
  const [tipo, setTipo] = useState('ESTUDIO')
```

Tareas.jsx

src > pages > Tareas.jsx > Tareas

```jsx
    }

    setTareas(tareas.filter(tarea => tarea.id !== id))
  }

  return (
    <main className="contenedor">
      <h1>Mi TODO List</h1>
      <p>Organizá tus tareas pendientes.</p>

      <form onSubmit={agregarTarea}>
        <input
          type="text"
          placeholder="Título de la tarea"
          value={titulo}
          onChange={event => setTitulo(event.target.value)}
        />
```

Hooks

useEffect

Permite ejecutar código cuando ocurre determinado **evento** relacionado con el ciclo de vida del componente, por ejemplo, cuando se muestra por primera vez.

https://www.w3schools.com/react/react_useeffect.asp

UseEffect

permite ejecutar código como consecuencia del renderizado de un componente, por ejemplo, para consultar una API, cargar datos o sincronizar información con un servicio externo.

En nuestro caso lo utilizamos para obtener las tareas desde el backend cuando se **carga la página.**

Tareas.jsx

src > pages > Tareas.jsx > Tareas

```jsx
import { useEffect, useState } from 'react'
import ListaTareas from '../components/ListaTareas'

const API_URL = 'http://localhost:8080/api/tareas'

function Tareas() {
  const [tareas, setTareas] = useState([])
  const [titulo, setTitulo] = useState('')
  const [descripcion, setDescripcion] = useState('')
  const [tipo, setTipo] = useState('ESTUDIO')

  useEffect(() => {
    fetch(API_URL)
      .then(response => {
        if (!response.ok) {
          throw new Error('Error al obtener las tareas')
        }
        return response.json()
      })
      .then(data => setTareas(data))
      .catch(error => console.error(error))
  }, [])
```

Renderizado condicional

Renderizado Condicional

El renderizado condicional en React consiste en mostrar diferentes elementos de la interfaz según se cumpla o no una condición.

Se puede implementar con **if**, el operador ternario (**? :**) o el operador lógico (**&&**).

https://www.w3schools.com/react/react_conditional_rendering.asp

Renderizado Condicional

En ListaTareas.jsx, mostramos un mensaje cuando no hay tareas.

¿Cómo funciona?

- Si tareas.length === 0 es verdadero, se muestra el mensaje “No hay tareas”.

- Si hay una o más tareas, el mensaje no se muestra.

ListaTareas.jsx

src > components > ListaTareas.jsx > ...

```jsx
import './ListaTareas.css'
import TareaItem from './TareaItem'

function ListaTareas({ tareas, onCambiarEstado, onEliminar }) {
  return (
    <section>
      <h2>Tareas ({tareas.length})</h2>

      {tareas.length === 0 && <p>No hay tareas.</p>}

      <div className="lista-tareas">
        {tareas.map(tarea => (
          <TareaItem
            key={tarea.id}
            tarea={tarea}
            onCambiarEstado={onCambiarEstado}
            onEliminar={onEliminar}
          />
        ))}
      </div>
    </section>
  )
}

export default ListaTareas
```

Router

SPA

React Router

Esta biblioteca permite definir rutas y mostrar distintos componentes según la URL.

```
npm install react-router-dom
```

https://www.w3schools.com/react/react_router.asp

Single Page Application

Una SPA (Single Page Application) es una aplicación web que carga una **única página HTML** y actualiza su contenido dinámicamente según las acciones del usuario, sin tener que recargar toda la página cada vez que navega.

SPA

De esta forma podemos implementar fácilmente una SPA con react.

App.jsx

src > App.jsx > ...

```jsx
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom'
import './App.css'
import Tareas from './pages/Tareas'
import AcercaDe from './pages/acercaDe'

function App() {
  return (
    <BrowserRouter>
      <nav className="navegacion">
        <Link to="/">Tareas</Link>
        <Link to="/acerca-de">Acerca de</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Tareas />} />
        <Route path="/acerca-de" element={<AcercaDe />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App
```

Fetch

Haciendo peticiones al backend

Fetch

fetch es una función de **JavaScript** que permite realizar solicitudes HTTP para enviar y recibir datos de un servidor, por ejemplo, para comunicarse con una API REST.

https://www.w3schools.com/js/js_api_fetch.asp

Agregar tarea

1. **Evita que se recargue la página**
2. **Valida el título**
3. **Prepara los datos**
4. **Envía la tarea al backend con fetch**
5. **await** espera a que llegue la respuesta del servidor antes de continuar
6. **Comprueba la respuesta**
7. **Actualiza la interfaz y limpia el formulario**
   - response.json() obtiene la tarea devuelta por el backend, que puede incluir el ID generado por la base de datos. Después, setTareas([...tareas, tareaGuardada]) agrega esa tarea al estado y React actualiza la lista. Finalmente, se limpian los campos del formulario.



CORS

Cross-Origin Resource Sharing

CORS

es un mecanismo de seguridad del navegador que controla qué sitios pueden hacer peticiones a un servidor. Como Vite se ejecuta en **localhost:5173** y Spring Boot en **localhost:8080**, el navegador considera que son orígenes diferentes.

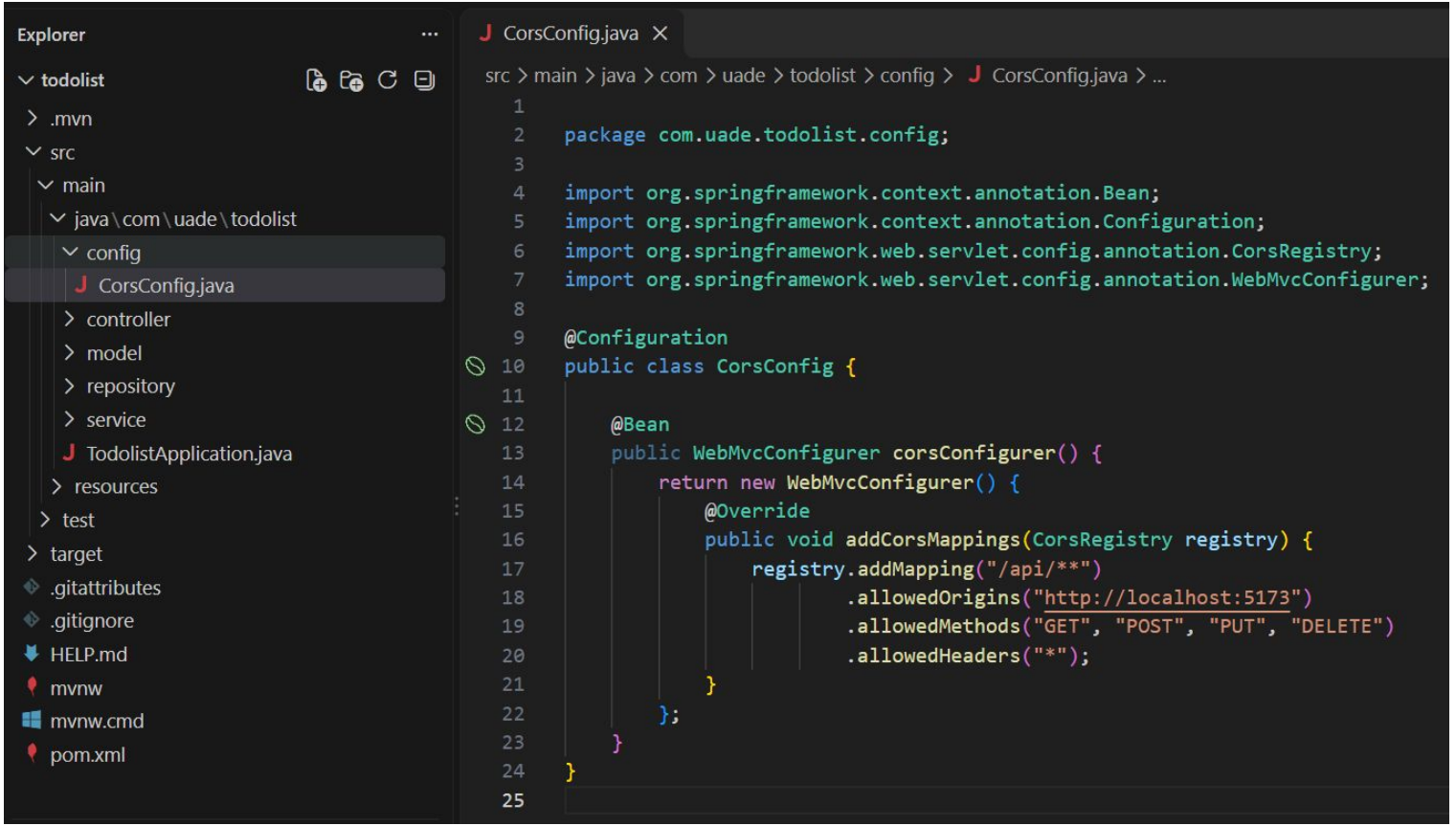
Los clientes que no son navegadores (como Postman, curl o peticiones directas de servidor a servidor) no aplican CORS, ignoran estas restricciones por completo.

CORS

Configuración

Para poder conectar el frontend con el backend, necesitamos registrar en nuestro servidor con spring aquellos clientes que pueden hacer peticiones HTTP.

CORS


Explorer
todolist
.mvn
src
main
java\com\uade\todolist
config
CorsConfig.java
controller
model
repository
service
TodolistApplication.java
resources
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
CorsConfig.java
src > main > java > com > uade > todolist > config > CorsConfig.java > ...
1 package com.uade.todolist.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.web.servlet.config.annotation.CorsRegistry;
6 import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
7
8 @Configuration
9 public class CorsConfig {
10
11 @Bean
12 public WebMvcConfigurer corsConfigurer() {
13 return new WebMvcConfigurer() {
14 @Override
15 public void addCorsMappings(CorsRegistry registry) {
16 registry.addMapping("/api/**")
17 .allowedOrigins("http://localhost:5173")
18 .allowedMethods("GET", "POST", "PUT", "DELETE")
19 .allowedHeaders("*");
20 }
21 };
22 }
23 }
24
25